

UNIVERSIDAD TECNOLÓGICA DE BOLÍVAR

Programa de Ingeniería de Sistemas y Computación

INFORME DE PRÁCTICA

Taller segundo corte: Implementación de una API con Docker y Docker Compose

Arquitectura de Software

Thiffany Torres Aparicio

Cod. T00082951

Luis Giovanny Carreno Ortiz

Cartagena de Indias, Colombia

Octubre de 2026

INTRODUCCIÓN

La práctica utiliza el repositorio biblioteca-api, una API desarrollada con Go y Gin que trabaja con MySQL. El procedimiento indicado por el taller consiste en revisar los archivos del proyecto, configurar las variables de entorno, construir e iniciar los servicios y comprobar la API. Las capturas finales deben tomarse en el computador del estudiante y demostrar los resultados reales.

Repositorio del proyecto: https://github.com/carrenoutb/biblioteca-api

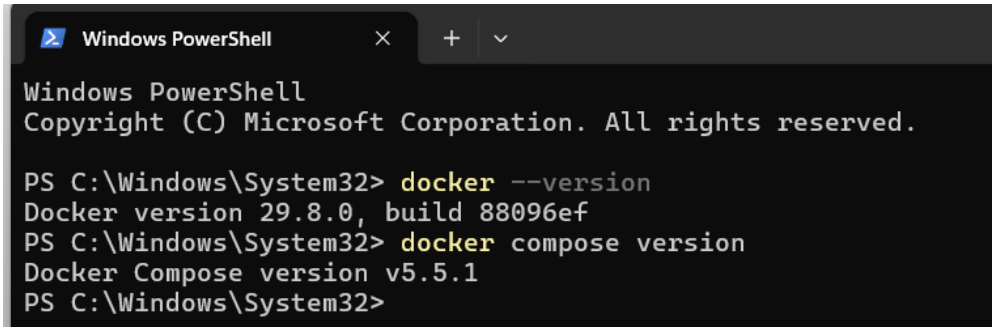
OBJETIVO

Ejecutar el proyecto biblioteca-api mediante Docker y Docker Compose, revisar sus archivos de configuración, comprobar el estado de los servicios y probar la API con Swagger. El informe registra tanto las comprobaciones realizadas como los problemas que se presentaron.

DESARROLLO DE LA PRÁCTICA

Verificación de Docker

Se comprobó la instalación con los comandos docker --version y docker compose version. En los resultados compartidos, el cliente Docker informó la versión 29.8.0 y Docker Compose la versión 5.5.1. También se ejecutó docker info, cuyo resultado incluyó información del servidor Docker Desktop, por lo que el motor respondió en el momento de esa comprobación.



Estado de WSL y Docker

Los resultados compartidos de wsl --status indicaron que Ubuntu era la distribución predeterminada y que la versión predeterminada era WSL 2. Con wsl --list --verbose se observó Ubuntu en estado Stopped y docker-desktop en estado Running, ambas con versión 2. En la salida de docker info se mostraron datos del servidor, kernel WSL2 y tres contenedores registrados: uno en ejecución y dos detenidos.

Revisión de los archivos

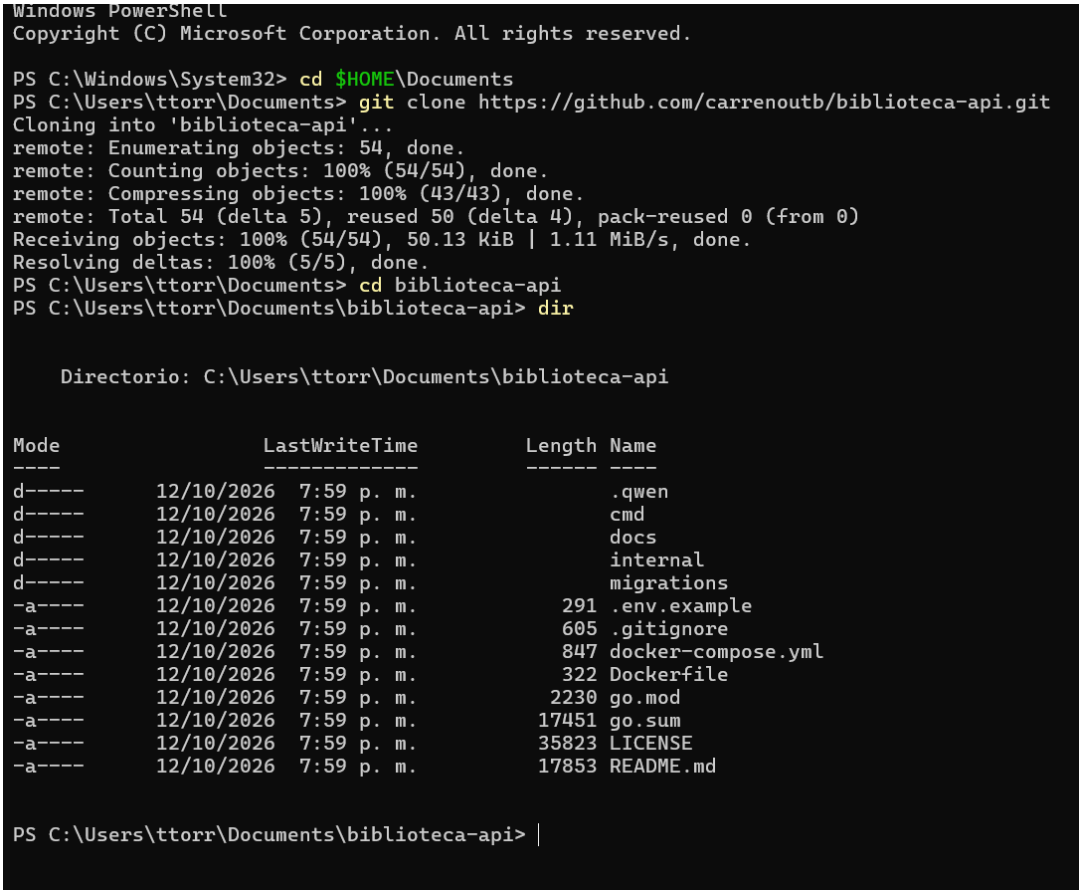
Dockerfile: contiene las instrucciones para construir la imagen de la aplicación.

docker-compose.yml: declara los servicios, sus relaciones y configuraciones de ejecución.

.env.example: sirve como plantilla de variables de entorno.

migrations/: reúne los archivos de migración de la base de datos.

Se debe confirmar el contenido concreto de cada archivo en el repositorio antes de describir detalles adicionales.



Configuración de variables de entorno

El README indica crear el archivo .env a partir de .env.example y revisar las variables de configuración. En PowerShell se puede utilizar Copy-Item .env.example .env si el archivo .env aún no existe. Si ya existe, no debe sobrescribirse sin revisar su contenido.

Variable	**Función**
DB_HOST	Nombre o dirección del servidor MySQL; debe coincidir con la red y el nombre del servicio configurado.
DB_PORT	Puerto de conexión a la base de datos.
DB_USER	Usuario que utiliza la aplicación para conectarse a MySQL.
DB_PASSWORD	Contraseña del usuario de base de datos.
DB_NAME	Nombre de la base de datos utilizada por la aplicación.
JWT_SECRET	Secreto utilizado por la aplicación para firmar o validar tokens JWT.
JWT_EXPIRATION_HOURS	Duración del token en horas.
APP_PORT	Puerto configurado para la aplicación o su publicación, según el proyecto.

Intento de construcción e inicio

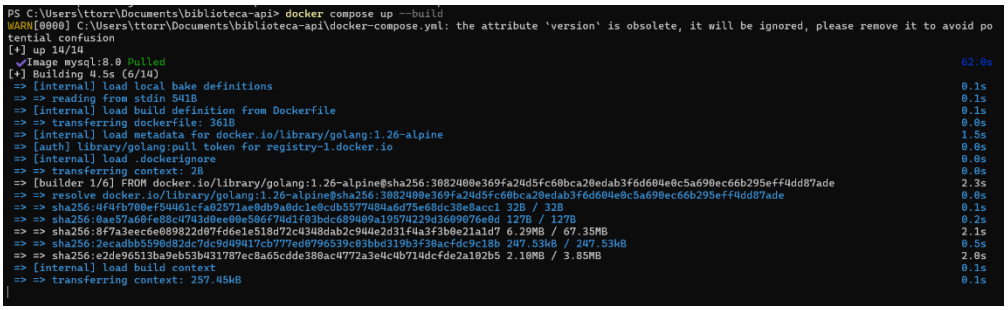
Al ejecutar docker compose up --build en la ruta C:\Users\ttorr\Documents\biblioteca-api, se recibió inicialmente el error indicando que no se podía conectar con npipe:////./pipe/dockerDesktopLinuxEngine y que el sistema no encontraba el archivo especificado.

La advertencia sobre el atributo versión obsoleto en docker-compose.yml también apareció; esa advertencia, por sí sola, no explica el error de conexión.

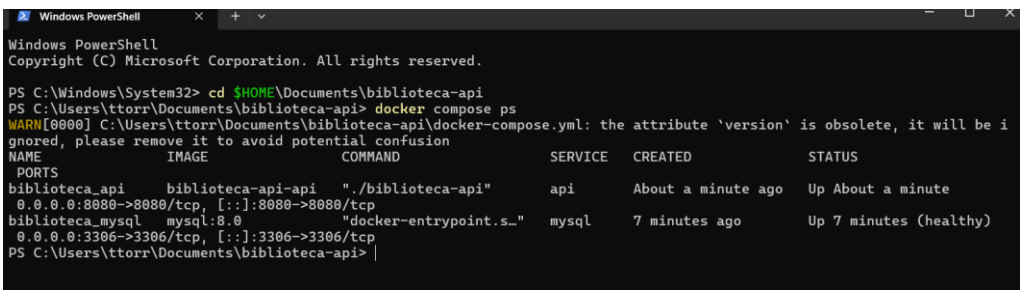
Después se consultaron los estados de WSL y docker info. La salida de docker info mostró información del servidor Docker Desktop, lo que indica que el motor respondió en ese momento. Más adelante, el comando docker compose ps mostró una tabla vacía para el proyecto; por tanto, con la evidencia disponible no se puede afirmar que los servicios de biblioteca-api hayan quedado iniciados correctamente.

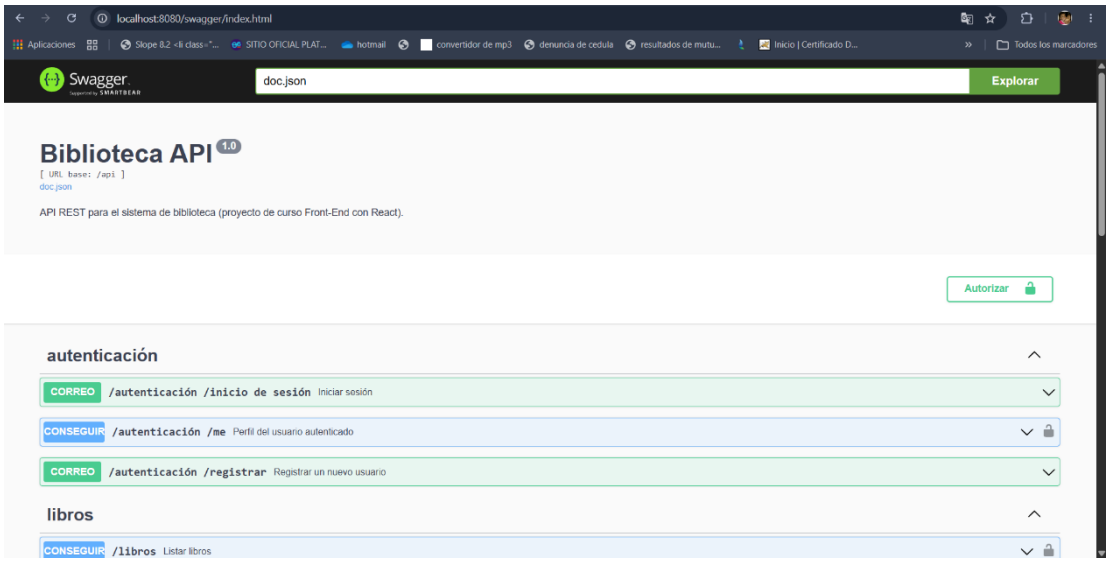
```
PS C:\Users\ttorr\Documents\biblioteca-api> docker compose up --build
WARN[0000] C:\Users\ttorr\Documents\biblioteca-api\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid po
tential confusion
[+] up 14/14
 ✔Image mysql:8.0 Pulled                                                                                                                                62.0s
[+] Building 4.5s (6/14)
 => [internal] load local bake definitions                                                                                                               0.1s
 => => reading from stdin 541B                                                                                                                           0.1s
 => [internal] load build definition from Dockerfile                                                                                                     0.1s
 => => transferring dockerfile: 361B                                                                                                                     0.0s
 => [internal] load metadata for docker.io/library/golang:1.26-alpine                                                                                    1.5s
 => [auth] library/golang:pull token for registry-1.docker.io                                                                                            0.0s
 => [internal] load .dockerignore                                                                                                                        0.0s
 => => transferring context: 2B                                                                                                                          0.0s
 => [builder 1/6] FROM docker.io/library/golang:1.26-alpine@sha256:3082400e369fa24d5fc60bca20edab3f6d604e0c5a690ec66b295eff4dd87ade                      2.3s
 => => resolve docker.io/library/golang:1.26-alpine@sha256:3082400e369fa24d5fc60bca20edab3f6d604e0c5a690ec66b295eff4dd87ade                              0.0s
 => => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1 32B / 32B                                                                 0.1s
 => => sha256:0ae57a60fe88c4743d0ee00e506f74d1f03bdc689409a19574229d3609076e0d 127B / 127B                                                               0.2s
 => => sha256:8f7a3eec6e089822d07fd6e1e518d72c4348dab2c944e2d31f4a3f3b0e21a1d7 6.29MB / 67.35MB                                                          2.1s
 => => sha256:2ecadbb5590d82dc7dc9d49417cb777ed0796539c03bbd319b3f30acfdc9c18b 247.53kB / 247.53kB                                                       0.5s
 => => sha256:e2de96513ba9eb53b431787ec8a65cdde380ac4772a3e4c4b714dcfde2a102b5 2.10MB / 3.85MB                                                           2.0s
 => [internal] load build context                                                                                                                        0.1s
 => => transferring context: 257.45kB                                                                                                                    0.1s
```

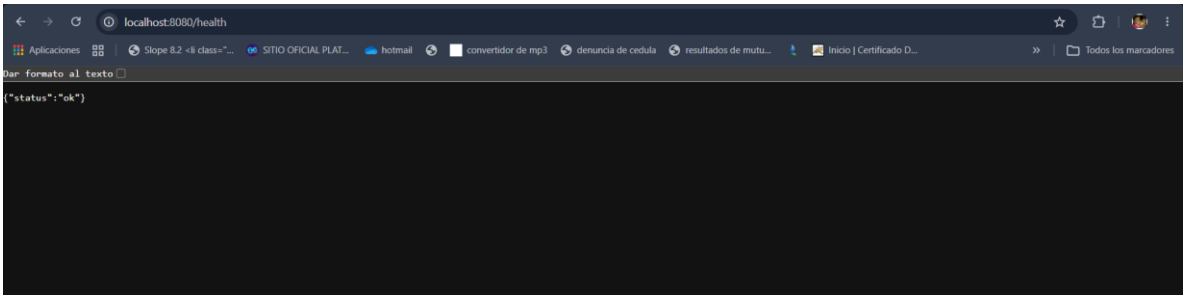
Verificación de servicios y Swagger

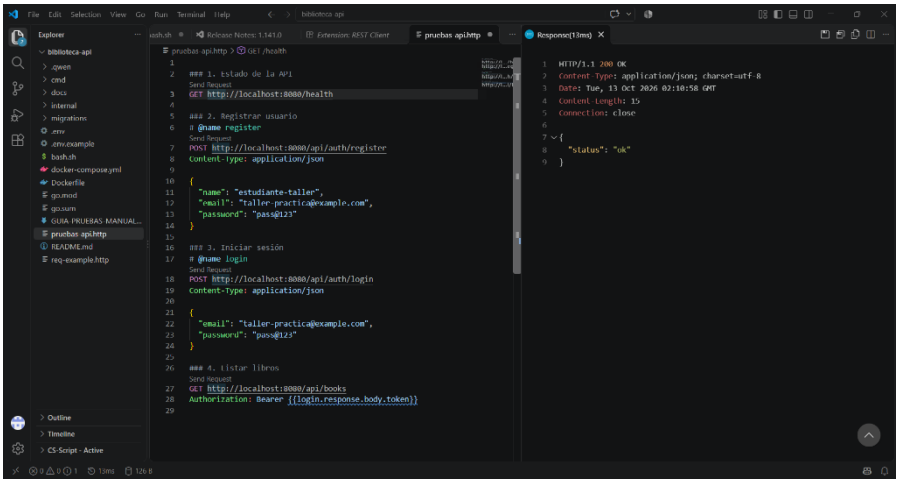


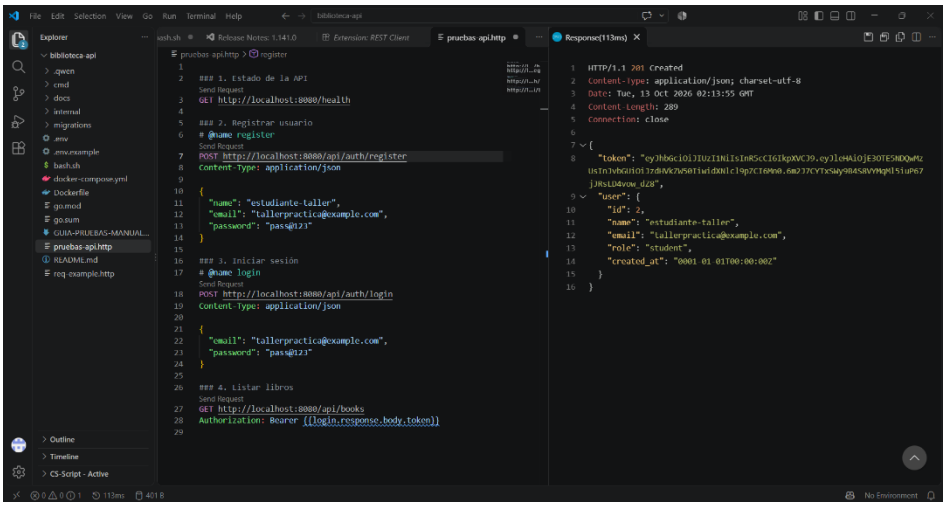

```
 => => transferring dockerfile: 361B                                                                                                                     0.0s
 => [internal] load metadata for docker.io/library/golang:1.26-alpine                                                                                    1.5s
 => [auth] library/golang:pull token for registry-1.docker.io                                                                                            0.0s
 => [internal] load .dockerignore                                                                                                                        0.0s
 => => transferring context: 2B                                                                                                                          0.0s
 => [builder 1/6] FROM docker.io/library/golang:1.26-alpine@sha256:3082400e369fa24d5fc60bca20edab3f6d604e0c5a690ec66b295eff4dd87ade                     19.9s
 => => resolve docker.io/library/golang:1.26-alpine@sha256:3082400e369fa24d5fc60bca20edab3f6d604e0c5a690ec66b295eff4dd87ade                              0.0s
 => => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1 32B / 32B                                                                 0.1s
 => => sha256:0ae57a60fe88c4743d0ee00e506f74d1f03bdc689409a19574229d3609076e0d 127B / 127B                                                               0.2s
 => => sha256:8f7a3eec6e089822d07fd6e1e518d72c4348dab2c944e2d31f4a3f3b0e21a1d7 67.35MB / 67.35MB                                                        12.8s
 => => sha256:2ecadbb5590d82dc7dc9d49417cb777ed0796539c03bbd319b3f30acfdc9c18b 247.53kB / 247.53kB                                                       0.5s
 => => sha256:e2de96513ba9eb53b431787ec8a65cdde380ac4772a3e4c4b714dcfde2a102b5 3.85MB / 3.85MB                                                           3.4s
 => => extracting sha256:e2de96513ba9eb53b431787ec8a65cdde380ac4772a3e4c4b714dcfde2a102b5                                                                 0.2s
 => => extracting sha256:2ecadbb5590d82dc7dc9d49417cb777ed0796539c03bbd319b3f30acfdc9c18b                                                                 0.1s
 => => extracting sha256:8f7a3eec6e089822d07fd6e1e518d72c4348dab2c944e2d31f4a3f3b0e21a1d7                                                                 6.8s
 => => extracting sha256:0ae57a60fe88c4743d0ee00e506f74d1f03bdc689409a19574229d3609076e0d                                                                 0.0s
 => => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1                                                                 0.0s
 => [internal] load build context                                                                                                                        0.1s
 => => transferring context: 257.45kB                                                                                                                    0.1s
 => [builder 2/6] WORKDIR /app                                                                                                                           0.6s
 => [builder 3/6] COPY go.mod go.sum* ./                                                                                                                 0.0s
 => [builder 4/6] RUN go mod download                                                                                                                   15.1s
 => [builder 5/6] COPY . .                                                                                                                               0.3s
 => [builder 6/6] RUN go build -o biblioteca-api ./cmd/api                                                                                              13.5s
 => [stage-1 3/4] COPY --from=builder /app/biblioteca-api .                                                                                              0.1s
 => [stage-1 4/4] COPY --from=builder /app/docs ./docs                                                                                                   0.0s
 => exporting to image                                                                                                                                   1.6s
 => => exporting layers                                                                                                                                  1.3s
 => => exporting manifest sha256:a70d3dbe68ab4bc432879c84a1e3bacced71342cb6b660f8cc5fde11bc686079                                                        0.0s
 => => exporting config sha256:4b8961481ec4be35949d321015bb3cb276096d03aa1474f479372d7d08d09840                                                          0.0s
 => => exporting attestation manifest sha256:b0a2f88dfba9fd2bc3f0303ddc0a049a098899add9247c8093717f6053a37cf7                                            0.0s
 => => exporting manifest list sha256:a742d6fe048572c7bebdd7480a3d5e510db772f3ee70670dd2050aac58716c9b                                                   0.0s
 => => naming to docker.io/library/biblioteca-api-api:latest                                                                                             0.0s
[+] up 19/19king to docker.io/library/biblioteca-api-api:latest                                                                                          0.2s
 ✔Image mysql:8.0                Pulled                                                                                                                 62.0s
 ✔Image biblioteca-api-api       Built                                                                                                                  53.7s
 ✔Network biblioteca-api_default Created                                                                                                                 0.1s
 ✔Volume biblioteca-api_mysql_data Created                                                                                                               0.0s
 ✔Container biblioteca_mysql     Created                                                                                                                 0.1s
 ✔Container biblioteca_api       Created                                                                                                                 0.1s
Attaching to biblioteca_api, biblioteca_mysql
```

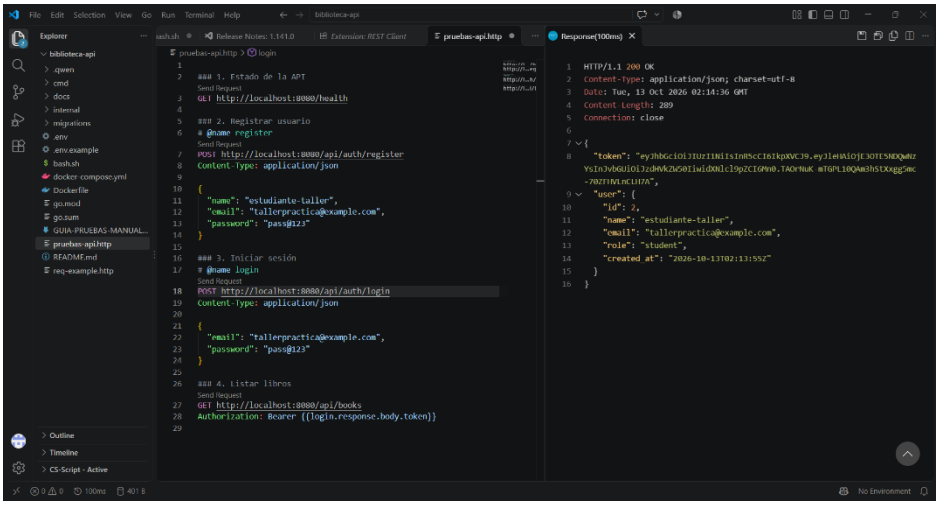


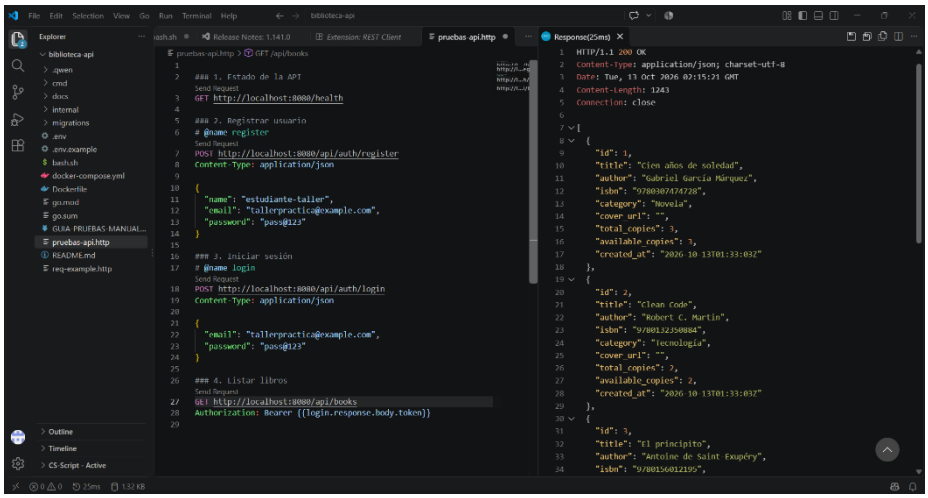

localhost:8080/swagger/index.html
Swagger
doc.json
Explorar
Biblioteca API
1.0
[URL base: /api]
doc.json
API REST para el sistema de biblioteca (proyecto de curso Front-End con React).
Autorizar
autenticación
CORREO /autenticación /inicio de sesión Iniciar sesión
CONSEGUIR /autenticación /me Perfil del usuario autenticado
CORREO /autenticación /registrar Registrar un nuevo usuario
libros
CONSEGUIR /libros Listar libros


localhost:8080/health
Dar formato al texto
{"status":"ok"}


pruebas api.http
1. Estado de la API
GET http://localhost:8080/health
2. Registrar usuario
POST http://localhost:8080/api/auth/register
Content-Type: application/json
{
"name": "estudiante-taller",
"email": "taller-practica@example.com",
"password": "pass@123"
}
3. Iniciar sesión
POST http://localhost:8080/api/auth/login
Content-Type: application/json
{
"email": "taller-practica@example.com",
"password": "pass@123"
}
4. Listar libros
GET http://localhost:8080/api/books
Authorization: Bearer {{login.response.body.token}}
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Tue, 13 Oct 2026 02:10:58 GMT
Content-Length: 15
Connection: close
{
"status": "ok"
}


pruebas-api.http
1. Estado de la API
GET http://localhost:8080/health
2. Registrar usuario
POST http://localhost:8080/api/auth/register
Content-Type: application/json
{
"name": "estudiante-taller",
"email": "tallerpractica@example.com",
"password": "pass@123"
}
3. Iniciar sesión
POST http://localhost:8080/api/auth/login
Content-Type: application/json
{
"email": "tallerpractica@example.com",
"password": "pass@123"
}
4. Listar libros
GET http://localhost:8080/api/books
Authorization: Bearer {{login.response.body.token}}
Response(113ms)
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Date: Tue, 13 Oct 2026 02:13:55 GMT
Content-Length: 289
Connection: close
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3OTE5MDQwMzUsInJvbGUiOiJzdHVkZW50IiwidXNlcl9pZCI6Mn0.6m2J7CYTxSWy9B4S8VYMqMlSiuP67jJRsLD4vow_d28",
"user": {
"id": 2,
"name": "estudiante-taller",
"email": "tallerpractica@example.com",
"role": "student",
"created_at": "0001 01 01T00:00:00Z"
}


pruebas-api.http
1. Estado de la API
GET http://localhost:8080/health
2. Registrar usuario
POST http://localhost:8080/api/auth/register
Content-Type: application/json
{
"name": "estudiante-taller",
"email": "tallerpractica@example.com",
"password": "pass@123"
}
3. Iniciar sesión
POST http://localhost:8080/api/auth/login
Content-Type: application/json
{
"email": "tallerpractica@example.com",
"password": "pass@123"
}
4. Listar libros
GET http://localhost:8080/api/books
Authorization: Bearer {{login.response.body.token}}
Response(100ms)
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Tue, 13 Oct 2026 02:14:36 GMT
Content-Length: 289
Connection: close
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3OTE5MDQwNzYsInJvbGUiOiJzdHVkZW50IiwidXNlcl9pZCI6Mn0.TAOrNuK_mTGPL10QAm3hStXxggSmc-70ZfIVLnCUI7A",
"user": {
"id": 2,
"name": "estudiante-taller",
"email": "tallerpractica@example.com",
"role": "student",
"created_at": "2026-10-13T02:13:55Z"
}


pruebas-api.http
1. Estado de la API
GET http://localhost:8080/health
2. Registrar usuario
POST http://localhost:8080/api/auth/register
Content-Type: application/json
{
"name": "estudiante-taller",
"email": "tallerpractica@example.com",
"password": "pass@123"
}
3. Iniciar sesión
POST http://localhost:8080/api/auth/login
Content-Type: application/json
{
"email": "tallerpractica@example.com",
"password": "pass@123"
}
4. Listar libros
GET http://localhost:8080/api/books
Authorization: Bearer {{login.response.body.token}}
Response(25ms)
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Tue, 13 Oct 2026 02:15:21 GMT
Content-Length: 1243
Connection: close
"id": 1,
"title": "Cien años de soledad",
"author": "Gabriel García Márquez",
"isbn": "9780307474728",
"category": "Novela",
"cover_url": "",
"total_copies": 3,
"available_copies": 3,
"created_at": "2026-10-13T01:33:032"
"id": 2,
"title": "Clean Code",
"author": "Robert C. Martin",
"isbn": "9/80132350884",
"category": "Tecnología",
"cover_url": "",
"total_copies": 2,
"available_copies": 2,
"created_at": "2026-10-13T01:33:032"
"id": 3,
"title": "El principito",
"author": "Antoine de Saint Exupéry",
"isbn": "9/80156012195",

PREGUNTAS DE COMPRENSIÓN

¿Qué diferencia hay entre una imagen y un contenedor?

Una imagen es la plantilla que contiene los archivos y metadatos para ejecutar una aplicación. Un contenedor es una instancia de esa imagen que se ejecuta. En biblioteca-api, una imagen de la API puede utilizarse para crear el contenedor donde corre el servicio.

¿Qué hace Docker Compose que no harías con docker run por separado?

Docker Compose permite definir y administrar varios servicios relacionados desde un archivo, incluyendo redes, variables, puertos y dependencias. Así se puede coordinar la API y MySQL sin tener que configurar cada contenedor por separado en comandos independientes.

¿Por qué la API encuentra la base de datos con DB_HOST=mysql y no con 127.0.0.1?

Si el servicio de la base de datos se llama mysql y ambos servicios comparten la red de Compose, la API puede usar ese nombre para encontrarlo. Dentro del contenedor de la API, 127.0.0.1 apunta al propio contenedor y no al contenedor de MySQL. Se debe confirmar que el nombre real del servicio en el archivo Compose sea mysql.

¿Qué borra exactamente docker compose down -v y cuándo lo usarías?

Detiene y elimina los contenedores del proyecto y las redes creadas por Compose; con -v también elimina los volúmenes nombrados declarados en el proyecto y los volúmenes anónimos asociados. Puede borrar datos persistentes, por lo que solo se debe usar cuando se desea eliminar esos datos y se ha comprobado qué volúmenes se afectarán.

¿Qué pasaría si cambias DB_PASSWORD en .env pero no en docker-compose.yml, o al revés?

Depende de cómo Compose lea las variables y de qué valor reciba cada servicio. Si la API intenta conectarse con una contraseña diferente de la que acepta MySQL, la autenticación puede fallar. Además, cambiar una variable no necesariamente cambia la contraseña de un usuario ya creado en una base de datos persistente. Se debe revisar cómo están declaradas las variables y cómo se inicializa la base de datos.

PROBLEMAS ENCONTRADOS Y POSIBLES SOLUCIONES

Problema 1: Docker Engine no disponible al ejecutar Compose.

El error informó que no podía conectarse al pipe dockerDesktopLinuxEngine. Se revisó WSL y se comprobó después que docker info sí devolvía información del servidor. El resultado compartido no confirma todavía que el comando docker compose up --build haya terminado correctamente; se debe volver a ejecutar y guardar el mensaje final para documentar la solución definitiva.

Problema 2: advertencia de la propiedad version.

Compose informó que el atributo version en docker-compose.yml está obsoleto y será ignorado. Esta advertencia no es por sí sola la causa de la desconexión del motor. Puede documentarse y, si se decide editar el archivo, debe hacerse solo después de revisar el proyecto y comprobar que la configuración continúa funcionando.

Otro problema posible es que las contraseñas configuradas para la API y MySQL no coincidan. La solución consiste en comparar las variables que recibe cada servicio y revisar los registros, sin publicar contraseñas ni borrar volúmenes de manera innecesaria.

CONCLUSIÓN

La práctica permite relacionar los conceptos de Docker con un proyecto que combina una API y una base de datos. Las comprobaciones realizadas muestran que Docker Desktop respondió a docker info después del error inicial, pero docker compose ps mostró una tabla vacía para el proyecto. Por eso, para considerar terminada la práctica aún falta confirmar la ejecución de los servicios, abrir Swagger y completar las operaciones de la API con capturas reales.

REFERENCIAS

carrenoutb. (s. f.). *biblioteca-api*. GitHub. https://github.com/carrenoutb/biblioteca-api

Docker. (s. f.). *Docker documentation*. https://docs.docker.com/

Docker. (s. f.). *Docker Compose.* https://docs.docker.com/compose/